DUTCH CLOUD NATIVE DAY · MISSION 2

We GitOps'd everything except the settings that break production.

Simon Koudijs



6 / 96 seats claimed



Simon Koudijs

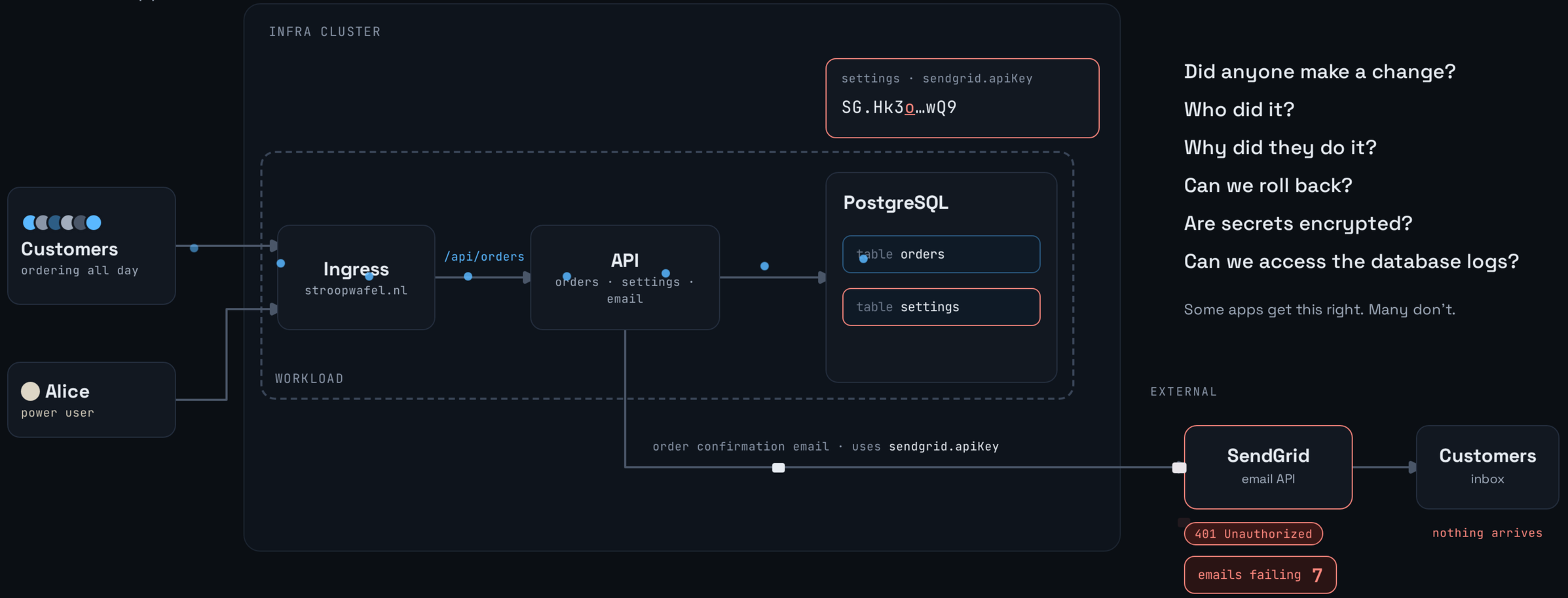
`[Software|Product|Cloud|AI] engineer`

- From the Netherlands
- Left my job in 2025 to build my own company
- **koudijs.dev**: consultancy, training, speaking
- **reversegitops.dev**: the pattern, and a manifesto
- **ConfigButler**: the hosted product of this idea

THE PROBLEM

One typo. No more customer emails.

Settings often live in the app's database, next to the orders. One small edit can break production, and it is not always clear what happened.



Did anyone make a change?

Who did it?

Why did they do it?

Can we roll back?

Are secrets encrypted?

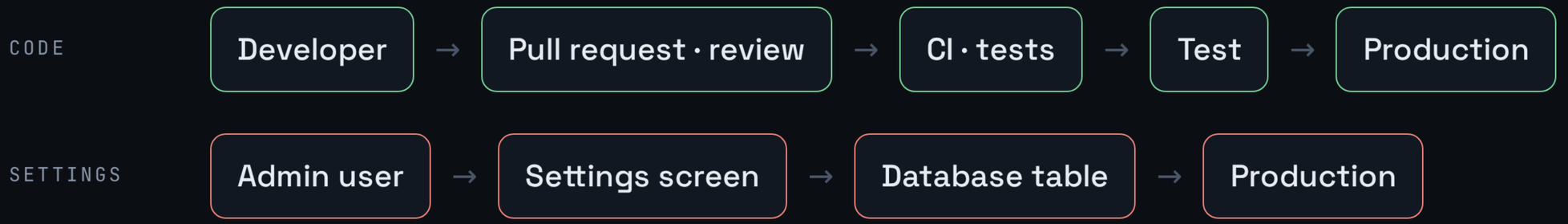
Can we access the database logs?

Some apps get this right. Many don't.

04 Some apps handle this well. Many don't, and you find out in production.

01 emails flow · 02 one typo · 03 silent failure · 04 no answers

Code has a delivery process. Settings don’t.



Feature flags, business rules, API keys and domain settings change production **just as much as code.**

WHAT WE WANT INSTEAD

A settings change should be...

- **recorded and reviewable**, like every other change to production
- **attributed**: you know exactly who did it
- **explained**: a short message on why somebody made it
- **reversible** in one step, without touching a database
- **visible** without production DB access

Isn’t this just a git commit?

My p n: Git as the database, my own API s op



The to-do list

- Who are you? `login, tokens, service account`
- ay you do this? `per type, per ob ction`
- Is d? `schemas, ru ts`
- Two edits `ing, conflicts`
- Tell m Cha `not polling`
- d what, when? `an audi`
- Clients `CLI, SDKs, a GUI`

Every item on that li ature of **kube-apiserver**. Already built, tested, and in produc er the world.

WHAT KUBE-APISERVER ALREADY DOES

Why build an API? This one is done.

Watch

`GET …/emailsettings?watch=true`

A live stream of every change. Apps, GUIs and Reverser react within milliseconds, no polling.

RBAC

`verbs: [get, update] · emailsettings`

Per kind, per namespace, per verb, even per object name. Support may read, admins may write.

Admission

`OpenAPI · CEL · webhooks`

Bad config is rejected before it is stored. Defaults are filled in. Your own rules in a policy.

Audit log

`user · verb · object · time`

Every request, with the real user on it. This is where Reverser gets the commit author.

Authentication

`OIDC · certs · service accounts`

The same login as the rest of your platform. Impersonation for tools that act on your behalf.

Concurrency

`resourceVersion → 409 Conflict`

Two people save at once? The second one gets a conflict, not a silent overwrite.

Discovery

`kubectl explain emailsettings`

Clients learn the schema at runtime. Forms and docs come from the CRD itself.

Ecosystem

`kubectl · client-go · Flux · MCP`

Every tool, library and AI agent that speaks Kubernetes already speaks your API.

So Git doesn't need to be the database. **kube-apiserver is the API.**

THE SAME API, FROM A TERMINAL

Your seats. Just a `kubectl get`.

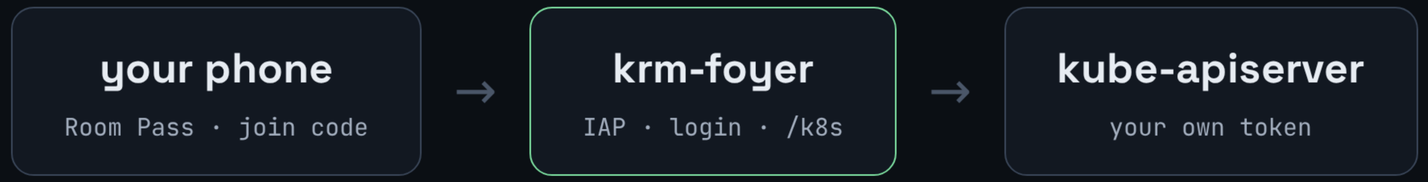
```
$ kubectl get seats -n voter
```

The hard parts

1. **Public access**
   getting people outside the cluster to the API, safely
2. **Modelling your application**
   building the CRDs
3. **Convincing people**
   a. architects
   b. developers
   c. cluster admins
4. **Explaining it to GitOps people**
   this is the source of truth

HARD PART 1 · PUBLIC ACCESS

You just did it. From your phone.



Your own token demo.koudijs.dev/auth/whoami

RBAC on a group demo.koudijs.dev/k8s/…/voter/seats

Admission

One watch for the room

OPEN SOURCE

krm-foyer
github.com/ConfigButler/krm-foyer
an identity-aware proxy (IAP) for the Kubernetes API

Room Pass
github.com/sunib/room-pass
OIDC with your audience: a code, a name, no accounts

Model it, and the API does the work.

```yaml
apiVersion: examples.configbutler.ai/v1alpha1
kind: Seat
metadata:
  name: mission2-6-6      # plan-row-column
spec:
  seatingPlanRef: { name: mission2 }
  row: 6                  # immutable
  column: 6               # immutable
  claimedBy:              # immutable
    displayName: simon-on-stage
    username: demo:Cg5zaW1v…
  color: '#a8e05f'        # may change
  emoji: 🐧               # one of 24
```

- **The seat is in the name.** Two phones claim one seat: the second gets `409 AlreadyExists`. That is the whole locking story
- **Fixed where it must be.** Row, column and owner never change: a CEL rule, `self == oldSelf`
- **A list, not free text.** 24 emoji that read well on a projector. Nothing unexpected gets through
- **Spec is the intent.** That is what goes to Git; secrets go in their own Secret

It’s real design work. But it’s **your app’s model**, not another settings table.

OPEN SOURCE

ConfigButler skills
`github.com/ConfigButler/skills`
homemade, for you and your AI agents

Is this infra?

PLATFORM TEAM OWNS IT

- Namespace
- Deployment
- Ingress
- Certificate

- Seat
- SeatingPlan

your place · the size of the room

- Feature flags
- Business rules
- API keys for external services

today: a database table

← →

APP TEAM OWNS IT

- Transaction data

stays in a database

kube-api doesn't care. It stores declarative desired state, whoever it belongs to. The line is a habit, not a limit.

READ MORE

Config Deserves Better
configdeservesbetter.dev
the pattern, zoomed out: 13 principles for config

Stop building settings screens. Ship a CRD.

Today, per app

- A settings table and a migration
- An admin screen with its own validation
- Who may change what: hand-rolled
- History and rollback: usually none

With kube-api, for free

- Schema validation: OpenAPI and CEL rules
- RBAC per resource, plus an audit log
- Read them live from the API, with a watch
- Or just as a file, in a mounted volume
- `kubectl`, any GUI, any script or AI agent

READ MORE

Config Deserves Better
configdeservesbetter.dev
the pattern, zoomed out: 13 principles for config

Not your workload cluster.
One per change, from production state.

“Strangers on my API server?”	A temporary, data-only kube-apiserver: nothing runs there. One per change, starting from the production state every time.
“Who may do what?”	RBAC per group, admission policies per kind. One RoleBinding to take it all back.
“How do the CRDs get there?”	Through Flux, like everything else you run. Reviewed in Git.
“What does it cost me?”	Small objects that change rarely, and one shared watch per kind. Plus: audit flags on the API server, for real authors.

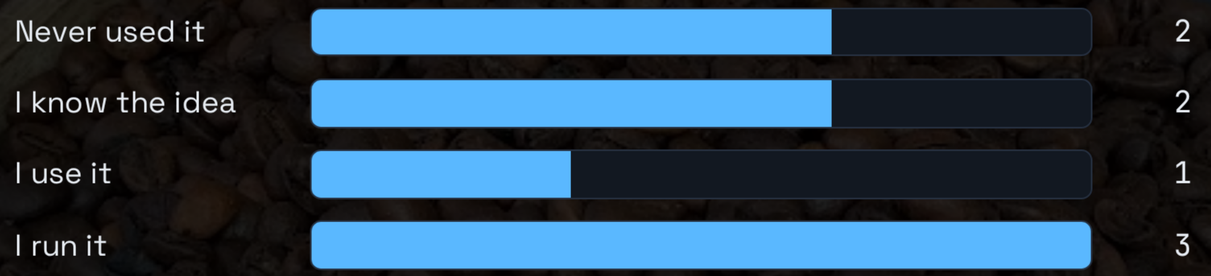
kcp
github.com/kcp-dev/kcp

Kamaji
github.com/clastix/kamaji

vCluster
github.com/loft-sh/vcluster

HARD PART 4 · EXPLAINING IT TO GITOPS PEOPLE

You know GitOps.



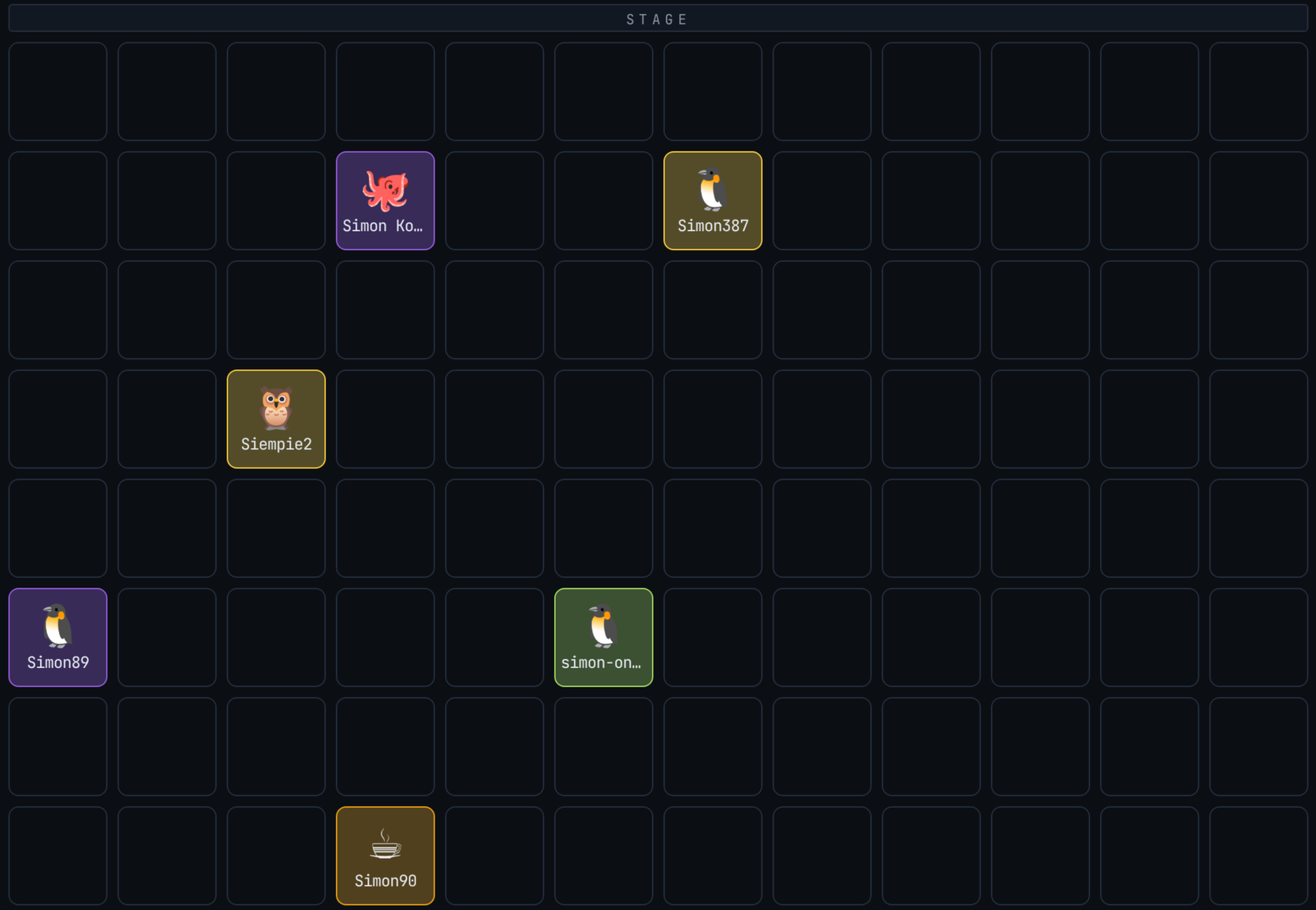
8 answered

> End users don’t need to be in Git.
> **But their actions should be.**

Here are your commits. With your names.

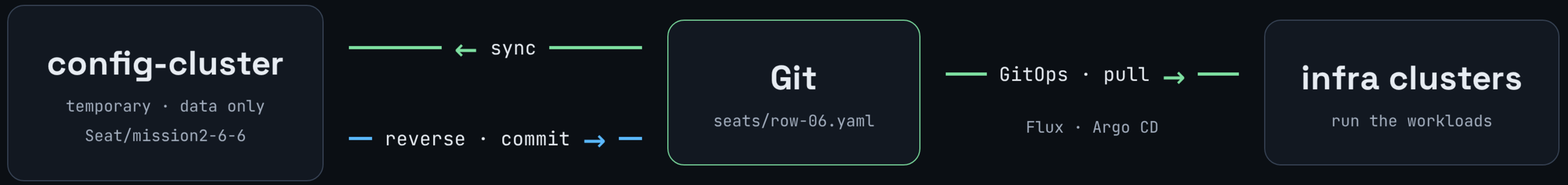
```
$ cd /workspaces/talks/external/k8s-trail
$ git pull --ff-only
$ git --no-pager log --format='%C(yellow)%h%C(reset) %C(cyan)%<(14,trunc)%an%C(reset) %s' -15
$ git --no-pager log -1 --stat --format=fuller
$ tree -L 2 -I README.md
$ yq seats/row-02.yaml
```

WHAT YOU JUST DID · YOUR SEAT IS A KUBERNETES OBJECT



6 / 96 seats claimed

One document. Both directions.



GitOps compatible. Infra clusters pull from Git, exactly as they do today

Editing happens through the temporary config-cluster. One per change, starting from production state

Bi-directional. Git to the API, and the API back to Git: nothing gets lost on the way, the object holds it all

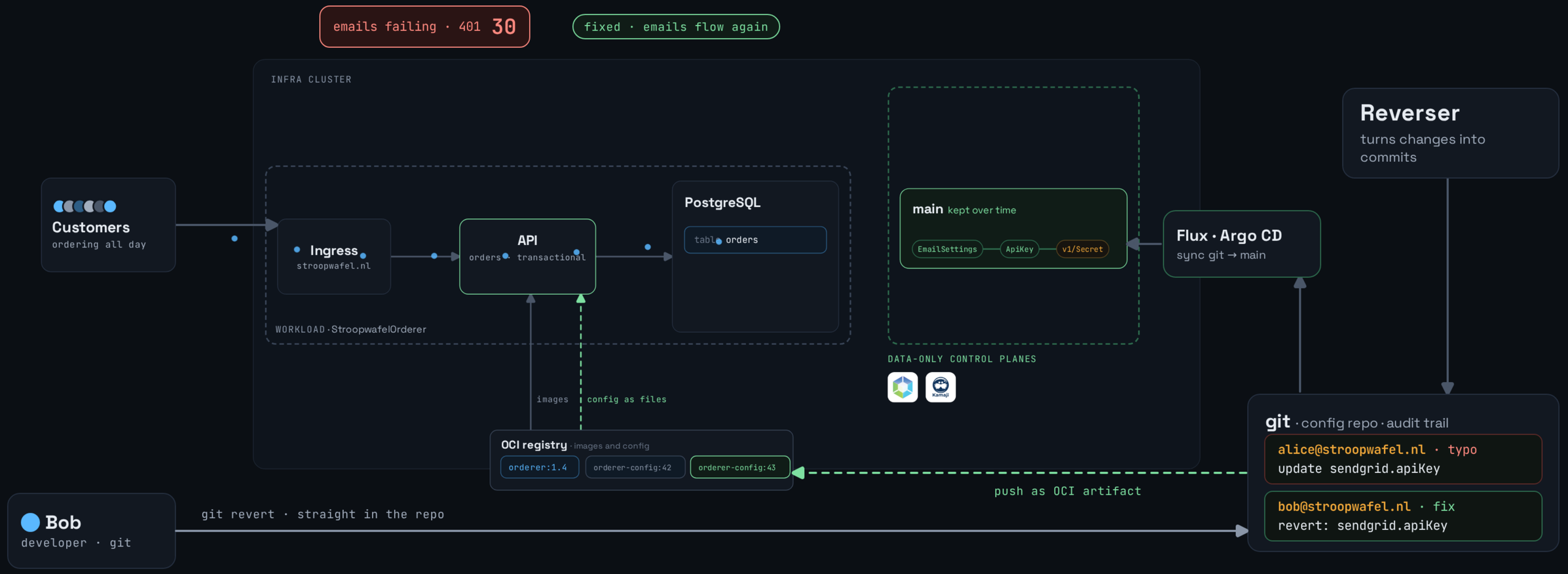
LINKS

GitOps Reverser
github.com/ConfigButler/gitops-reverser
the way back: API resources into Git as clean YAML, commits as the audit trail

Reverse GitOps
reversegitops.dev
the operating model: change through the API, a controller writes it to Git

Settings are data. Give them their own home.

Orders stay in the app. Settings move behind a data-only API that users never notice, and from there into Git.



08 Bob reverts in Git. The fix ships to the app, the emails flow again, and the next config cluster starts from it.

01 today · 02 split · 03 reads · 04 platform · 05 same typo · 06 deploy · 07 cleanup · 08 revert

DEMO

You are the admins.
Change the coffee menu.

Pros and cons

Pros

- People keep their settings screen; nobody needs to learn Git
- Structured, validated changes: schema, admission policies
- History with real authors, review and `git revert`
- The same API for people, scripts and AI agents, with the same limits

Cons

- Extra infrastructure: a config cluster plus Reverser
- Git runs a few seconds behind (commit windows)
- Conflicts: on the same resource, the API wins over a Git edit
- Real authors need audit flags on kube-apiserver: most managed control planes can’t
- PRs are not created yet

ONE ASK

What’s the ingredient I’m missing?

Connect with me, and tell me **one more ingredient.**
Critical questions are the most welcome.

I won’t spam you. I’d just love to hear from you.

The most valuable thing for me: people who give feedback.

CONNECT · LINKEDIN



linkedin.com/in/simonkoudijs

OR ⭐ STAR IT



gitops-reverser

BACKUP

How it works inside

Design it so it can remember.

Config as Kubernetes resources (KRM): not just the value, but the why, the expiry, the last test.

EmailSettings · StroopwafelOrderer config, in KCP

```
kind: EmailSettings
metadata: { name: order-confirmations }
spec:
  sender: orders@stroopwafel.nl
  templateId: order-confirmed
  apiKeyRef: sendgrid
```

ApiKey · its own CRD, in KCP

```
apiVersion: orderer.stroopwafel.nl/v1
kind: ApiKey
metadata:
  name: sendgrid
spec:
  secretRef:
    name: sendgrid-api-key
  scopes: [mail.send]
  expiresAt: "2026-11-20"
  requestedIn: OPS-1234
  owner: team-checkout
status:
  lastTestedAt: "2026-10-30T13:55:00Z"
  lastSuccessfulUseAt: "2026-10-30T14:01:12Z"
  conditions:
  - type: Ready
    status: "True"
  - type: ExpiringSoon
    status: "True"
    message: Expires in 21 days
```

in Git · reviewed

controller · never in Git

Secret · other RBAC, other eyes — in Git · SOPS

```
apiVersion: v1
kind: Secret
metadata: { name: sendgrid-api-key }
stringData:
  apiKey: ENC[AES256_GCM,data:q9Xr…]
sops: { gcp_kms: …/orderer-config }
```

Pod · orderer

`serviceAccountName: orderer · workload identity`

decrypt

key

Cloud KMS · key orderer-config

only the app's identity may decrypt

the plaintext key only exists in the app's memory

spec · what we *meant*: rights, expiry, the ticket

status · what *happened*: last tested, last used

the why · in the commit message

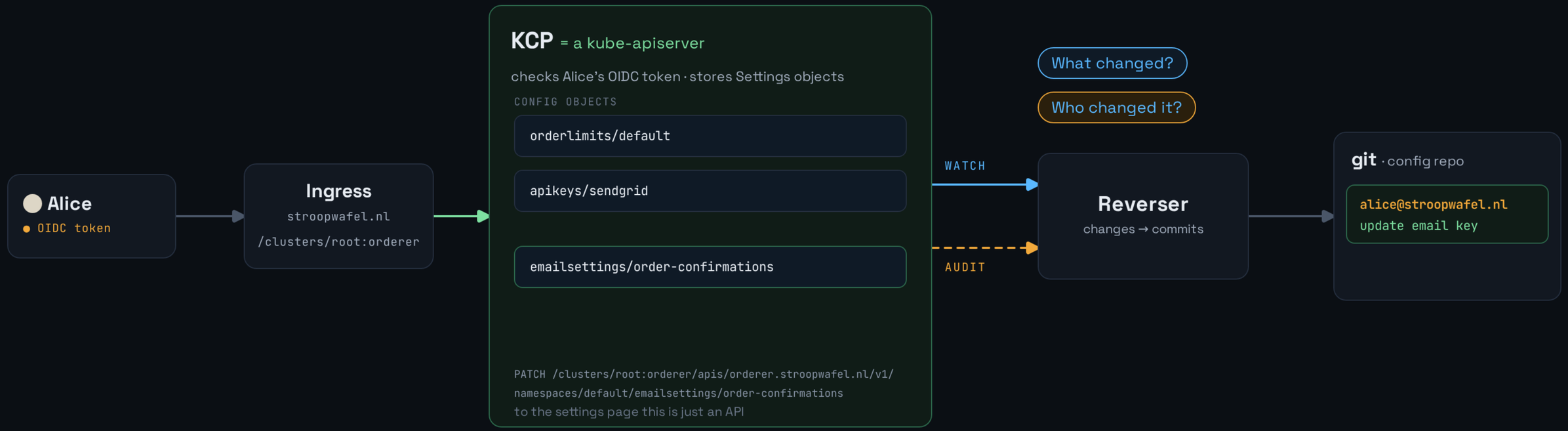
the value · lives in a Secret: other RBAC, other eyes

04 The app decrypts with its own workload identity. Anyone may read the ApiKey; only the app reads the key.

01 the rule · 02 what we meant · 03 what happened · 04 decrypt

KCP speaks Kubernetes. Reverser listens.

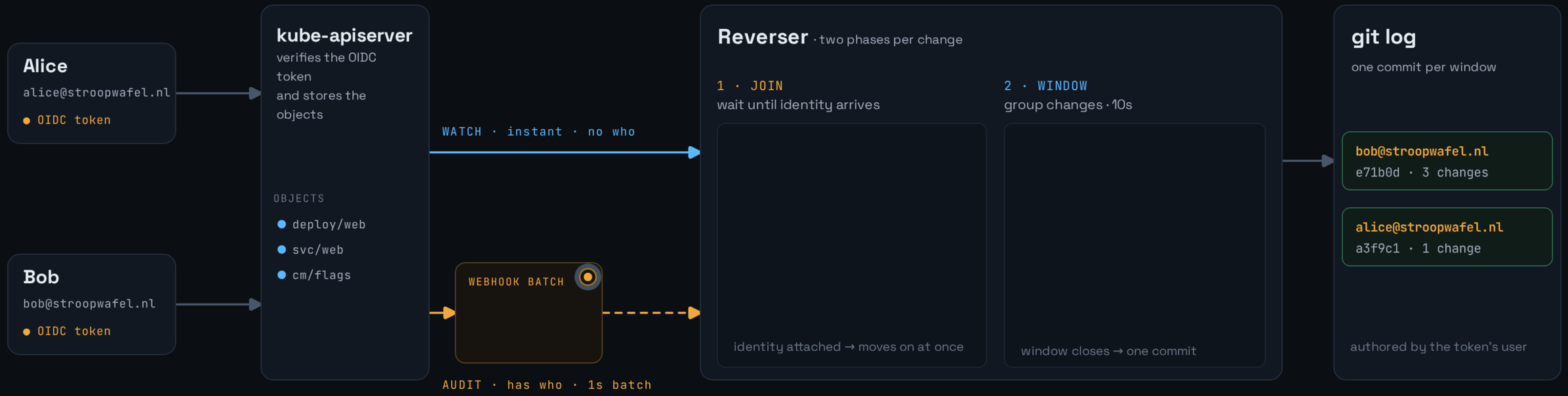
Settings are Kubernetes objects. Every change goes through an apiserver that knows exactly who you are.



04 The apiserver already offers both, as two separate streams. Let's look inside.

Two streams in. What changed, joined with who changed it.

The watch stream says what changed; the audit webhook says who. Each change waits only until its author is known, then joins a window. One window becomes one commit.

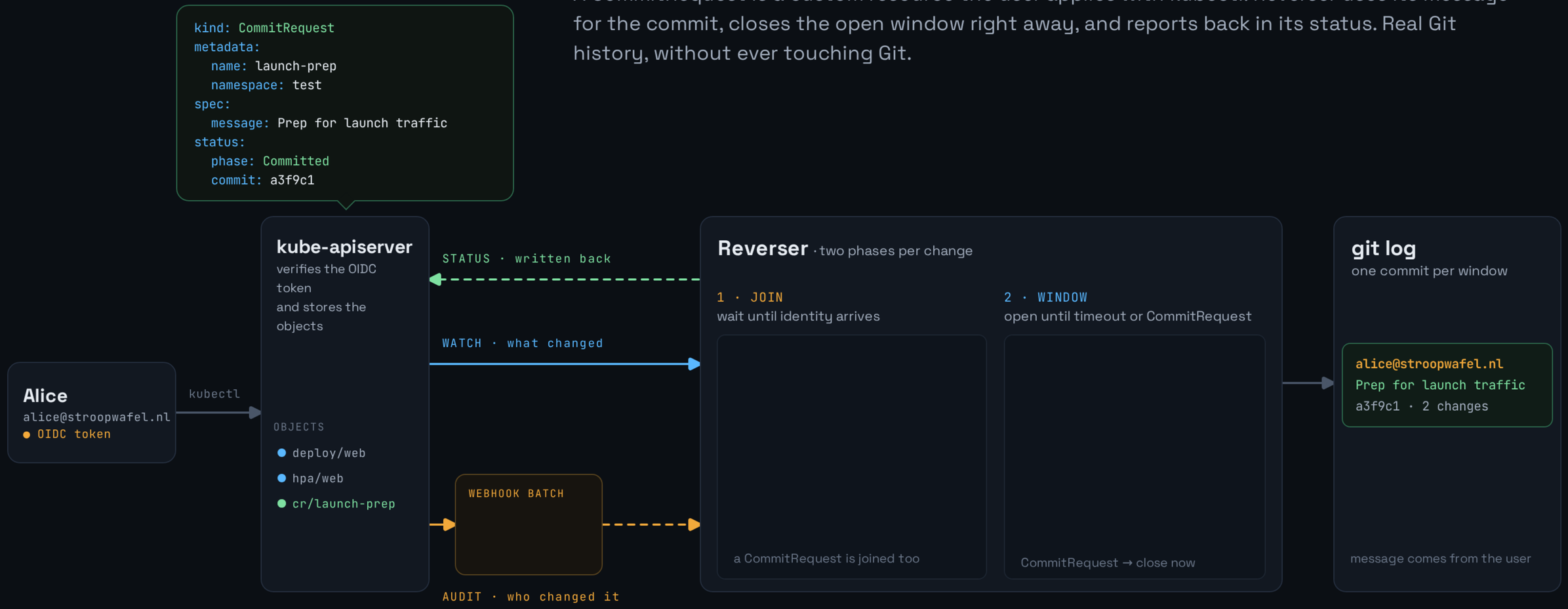


05 One commit by bob@stroopwafel.nl with all three: an update, a create and a delete.

01 change · 02 joined · 03 commit · 04 Bob: 3 changes · 05 commit

Say why, and the commit carries it.

A CommitRequest is a custom resource the user applies with kubectl. Reverser uses its message for the commit, closes the open window right away, and reports back in its status. Real Git history, without ever touching Git.



04 One commit by Alice with her reason as the message. The status now points to that commit.

01 changes · 02 CommitRequest · 03 window closed · 04 committed

Ship settings like images.

Settings ship as an OCI artifact. Flux picks up new versions, and the Deployment mounts them as a read-only image volume.

Inside orderer-config:43 · mounted at /etc/orderer/config

```
/etc/orderer/config/
├── emailsettings/order-confirmations.yaml
├── apikey/sendgrid.yaml                      # spec only, no status
└── secret/sendgrid-api-key.sops.yaml         # still SOPS-encrypted

# emailsettings/order-confirmations.yaml
spec: { sender: orders@stroopwafel.nl, templateId: order-confirmed, apiKeyRef: sendgrid }
```

The app · just reads YAML, through a typed lib generated from the CRDs

```
cfg := ordererconfig.Load("/etc/orderer/config")      // generated from the CRDs
email := cfg.EmailSettings("order-confirmations")
key := cfg.ApiKey(email.APIKeyRef).Decrypt(ctx)   // workload identity → KMS
mailer.Send(email.Sender, email.TemplateID, key, order)

// no API calls, no database: settings are just files
```

Deployment · orderer · in the infra cluster

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orderer
spec:
  template:
    spec:
      containers:
      - name: orderer
        image: ghcr.io/stroopwafel/orderer:1.4
        volumeMounts:
        - name: config
          mountPath: /etc/orderer/config
      volumes:
      - name: config
        image:  # OCI artifact as a volume
          reference: ghcr.io/stroopwafel/orderer-config:43 # {"$imagepolicy": "flux-system:config"}
          pullPolicy: IfNotPresent
```

rollout · new pod mounts orderer-config:43 read-only at /etc/orderer/config

05 And the app just reads YAML files. A typed library, generated from the CRDs, makes that a few lines of code.

01 publish · 02 registry · 03 Flux bumps · 04 mounted · 05 the app reads it